

中山大學
SUN YAT-SEN UNIVERSITY

大数据原理与技术 | 实验三

聚类算法对比实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 3 月 24 日

结 束 日 期: 2026 年 3 月 24 日

实 验 地 点: B201

目录

1 实验要求 **2**

2 算法原理 **2**

2.1 K-means 聚类算法 2

2.2 DBSCAN 聚类算法 2

3 代码展示 **3**

3.1 K-means 核心实现 3

3.2 DBSCAN 核心实现 4

3.3 准确率计算（Hungarian Algorithm） 4

4 结果分析 **6**

4.1 K-means 聚类结果 6

4.2 DBSCAN 聚类结果 7

4.3 综合对比 8

4.4 定量评估结果 8

4.5 结果分析 9

5 实验总结 **9**

1 实验要求

在 Iris（鸢尾花）数据集上手动实现 K-means 和 DBSCAN 算法，通过调整超参数观察结果变化，并使用定量指标评估性能。具体要求如下：

1. **算法实现：**禁止直接调用 sklearn.cluster.KMeans 或 sklearn.cluster.DBSCAN，需手动实现核心逻辑。
2. **K-means 实现：**初始化质心（建议使用 K-means++ 或随机）、距离计算（欧氏距离）、样本归类、质心更新循环（直至收敛或达到最大迭代次数）。
3. **DBSCAN 实现：**邻域搜索（ϵ-neighborhood）、核心点识别、簇增长逻辑（使用队列或递归实现密度相连点的扩展）。
4. **参数对比实验：**
   - K-means：尝试 $k = [2, 3, 4, 5]$
   - DBSCAN：固定 $MinPts = 5$，调整 $\epsilon = [0.2, 0.5, 0.8, 1.2]$
5. **评价指标：**准确率（需实现 Hungarian Algorithm 进行标签映射）、轮廓系数、Calinski-Harabasz Index。
6. **可视化：**生成不同参数下的散点图对比。

2 算法原理

2.1 K-means 聚类算法

K-means 是一种基于划分的聚类算法，目标是将 n 个样本划分为 k 个簇，使得簇内平方和最小化。其损失函数为：

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2 \tag{1}$$

其中，C_i 表示第 i 个簇，μ_i 表示第 i 个簇的质心。

算法流程：

1. 初始化 k 个质心（本实验采用 K-means++ 方法）
2. 计算每个样本到各质心的欧氏距离，分配到最近的簇
3. 重新计算每个簇的质心
4. 重复步骤 2-3 直到收敛或达到最大迭代次数

2.2 DBSCAN 聚类算法

DBSCAN 是一种基于密度的聚类算法，能够发现任意形状的簇并识别噪声点。

核心概念：

- **ϵ-邻域：**样本 x 的 ϵ-邻域为 $N_\epsilon(x) = \{y \in D \mid \|x - y\| \leq \epsilon\}$

- **核心点**：若 $|N_\epsilon(x)| \geq \text{MinPts}$，则 x 为核心点
- **噪声点**：既非核心点也非边界点的点

算法流程：

1. 对每个未访问的点，查询其 ϵ-邻域
2. 若邻域内点数 $\geq$ MinPts，则创建新簇并扩展
3. 使用队列实现簇扩展（广度优先搜索）
4. 标记无法被任何簇包含的点为噪声点

3 代码展示

3.1 K-means 核心实现

```python
class KMeansManual:
    def __init__(self, n_clusters=3, max_iter=300, tol=1e-4, random_state=42):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
        self.tol = tol
        self.random_state = random_state

    def _kmeans_plus_plus_init(self, X):
        """K-means++ 初始化质心"""
        np.random.seed(self.random_state)
        centroids = np.zeros((self.n_clusters, X.shape[1]))
        centroids[0] = X[np.random.randint(0, X.shape[0])]
        for k in range(1, self.n_clusters):
            distances = np.min([np.sum((X - centroids[j])**2, axis=1)
                                for j in range(k)], axis=0)
            prob = distances / distances.sum()
            centroids[k] = X[np.random.choice(X.shape[0], p=prob)]
        return centroids

    def fit(self, X):
        self.centroids = self._kmeans_plus_plus_init(X)
        for _ in range(self.max_iter):
            labels = self._assign_clusters(X)
            new_centroids = self._update_centroids(X, labels)
            if np.sum((new_centroids - self.centroids)**2) < self.tol:
                break
            self.centroids = new_centroids
        self.labels = labels
        return self
```

3.2 DBSCAN 核心实现

```python
class DBSCANManual:
    def __init__(self, eps=0.5, min_samples=5):
        self.eps = eps
        self.min_samples = min_samples

    def _region_query(self, X, point_idx):
        """查询 epsilon-邻域"""
        distances = np.sqrt(np.sum((X - X[point_idx])**2, axis=1))
        return np.where(distances <= self.eps)[0]

    def _expand_cluster(self, X, point_idx, neighbors, cluster_id,
                        visited, cluster_labels):
        """使用队列扩展簇"""
        cluster_labels[point_idx] = cluster_id
        queue = list(neighbors)
        while queue:
            current_idx = queue.pop(0)
            if not visited[current_idx]:
                visited[current_idx] = True
                current_neighbors = self._region_query(X, current_idx)
                if len(current_neighbors) >= self.min_samples:
                    for n in current_neighbors:
                        if cluster_labels[n] == -1:
                            queue.append(n)
            if cluster_labels[current_idx] == -1:
                cluster_labels[current_idx] = cluster_id

    def fit(self, X):
        n_samples = X.shape[0]
        visited = np.zeros(n_samples, dtype=bool)
        cluster_labels = np.full(n_samples, -1)
        cluster_id = 0
        for i in range(n_samples):
            if visited[i]: continue
            visited[i] = True
            neighbors = self._region_query(X, i)
            if len(neighbors) >= self.min_samples:
                self._expand_cluster(X, i, neighbors, cluster_id,
                                     visited, cluster_labels)
                cluster_id += 1
        self.labels = cluster_labels
        return self
```

3.3 准确率计算（Hungarian Algorithm）

```python
def compute_accuracy(y_true, y_pred):
    """使用 Hungarian Algorithm 进行标签映射后计算准确率"""
```

```
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    true_labels = np.unique(y_true)
    pred_labels = np.unique(y_pred[y_pred >= 0])

    # 构建代价矩阵
    cost_matrix = np.zeros((len(true_labels), len(pred_labels)))
    for i, tl in enumerate(true_labels):
        for j, pl in enumerate(pred_labels):
            cost_matrix[i, j] = -np.sum((y_true == tl) & (y_pred == pl))

    # Hungarian Algorithm 求解最优匹配
    row_ind, col_ind = linear_sum_assignment(cost_matrix)

    # 计算准确率
    total_correct = sum(np.sum((y_true == true_labels[i]) &
                               (y_pred == pred_labels[j]))
                        for i, j in zip(row_ind, col_ind))
    return total_correct / len(y_true)
```

4　结果分析

4.1　K-means 聚类结果

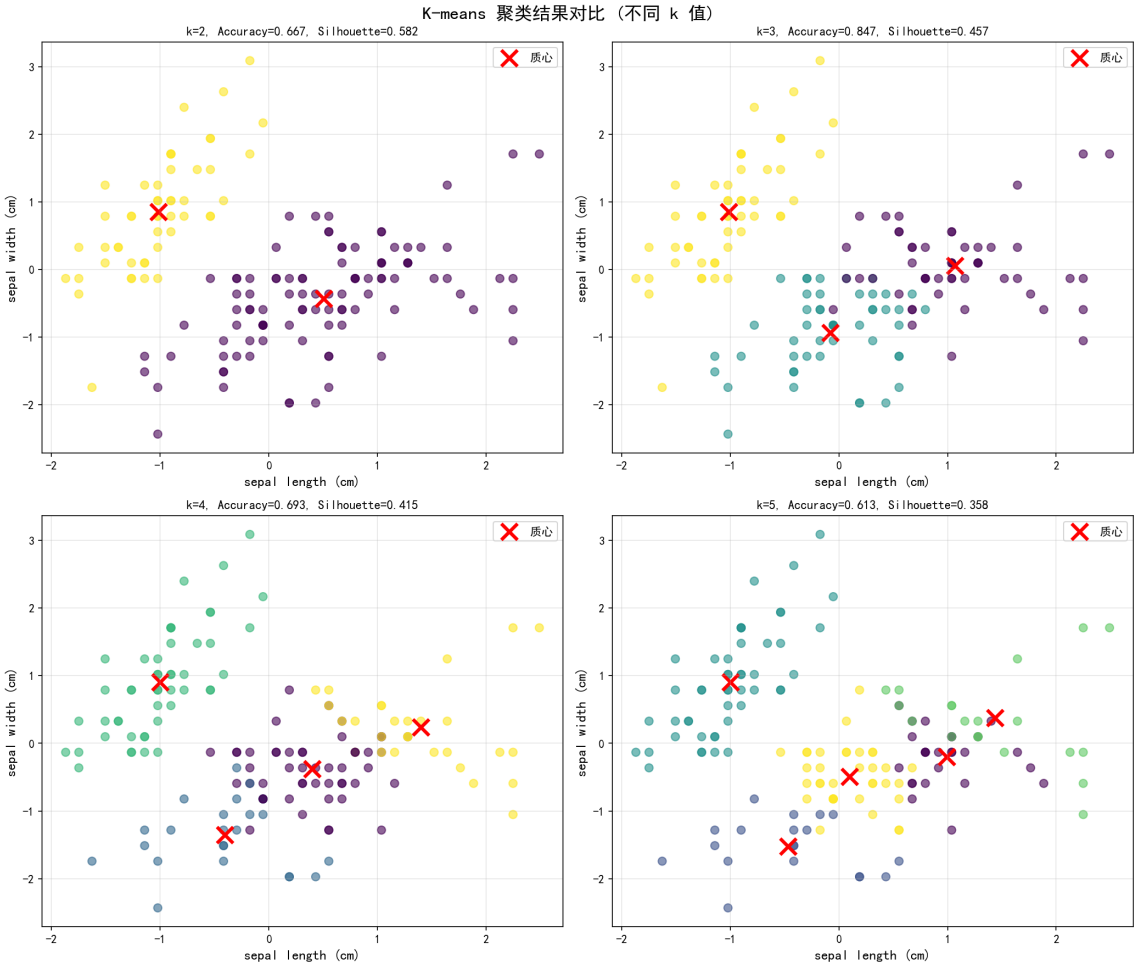


图 1: K-means 聚类结果对比（不同 k 值）

图 1 展示了 K-means 算法在不同 k 值下的聚类结果：

- 当 $k = 2$ 时，轮廓系数较高（0.5818），但准确率较低（66.67%）
- 当 $k = 3$ 时，与真实类别数一致，准确率达到最高（84.67%）
- 当 $k = 4$ 和 $k = 5$ 时，出现过度分割，准确率下降

4.2 DBSCAN 聚类结果

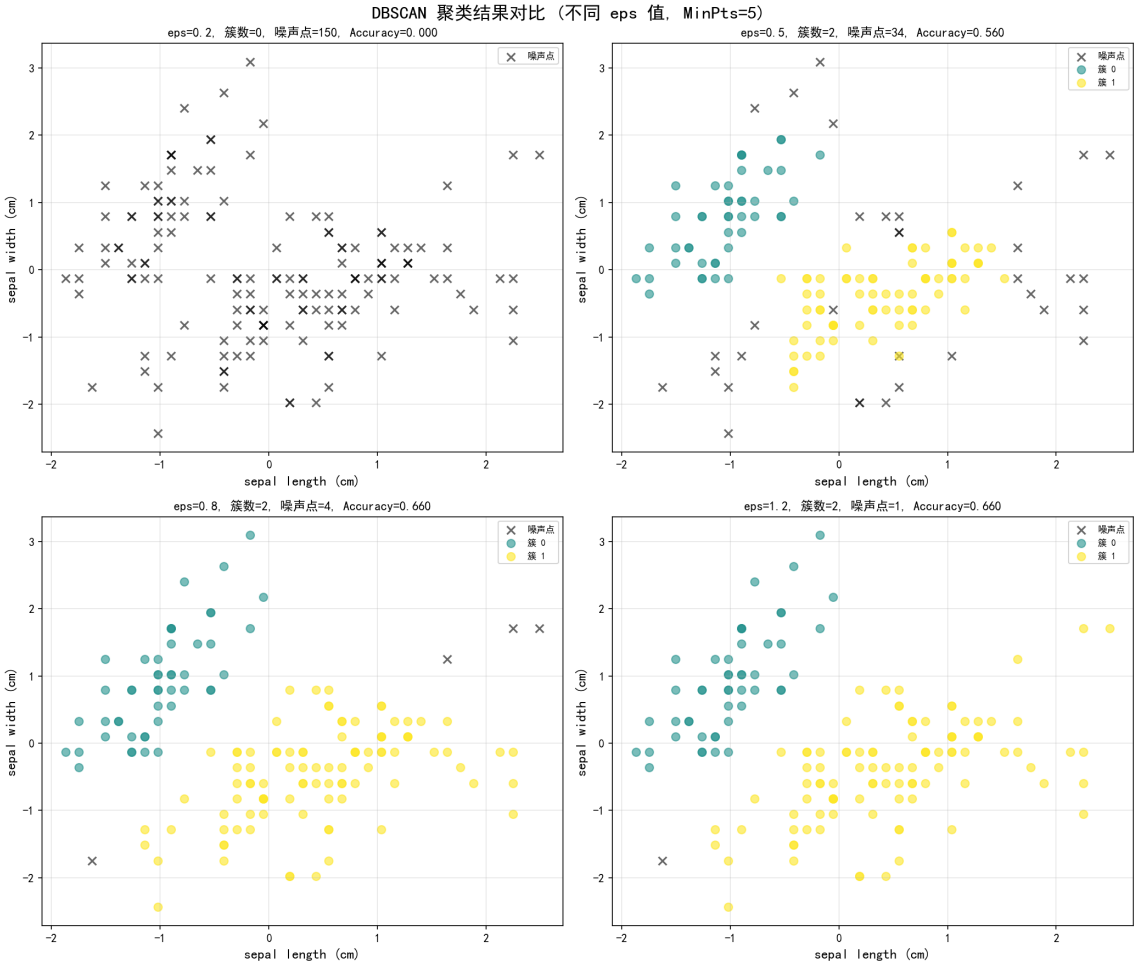


图 2: DBSCAN 聚类结果对比（不同 ϵ 值，MinPts=5）

图 2 展示了 DBSCAN 算法在不同 ϵ 值下的聚类结果（黑色"x" 表示噪声点）：

- 当 $\epsilon = 0.2$ 时，邻域过小，所有点都被识别为噪声点
- 当 $\epsilon = 0.5$ 时，识别出 2 个簇，34 个噪声点
- 当 $\epsilon = 0.8$ 时，识别出 2 个簇，仅 4 个噪声点，准确率 66.00%
- 当 $\epsilon = 1.2$ 时，邻域过大，仅 1 个噪声点

4.3 综合对比

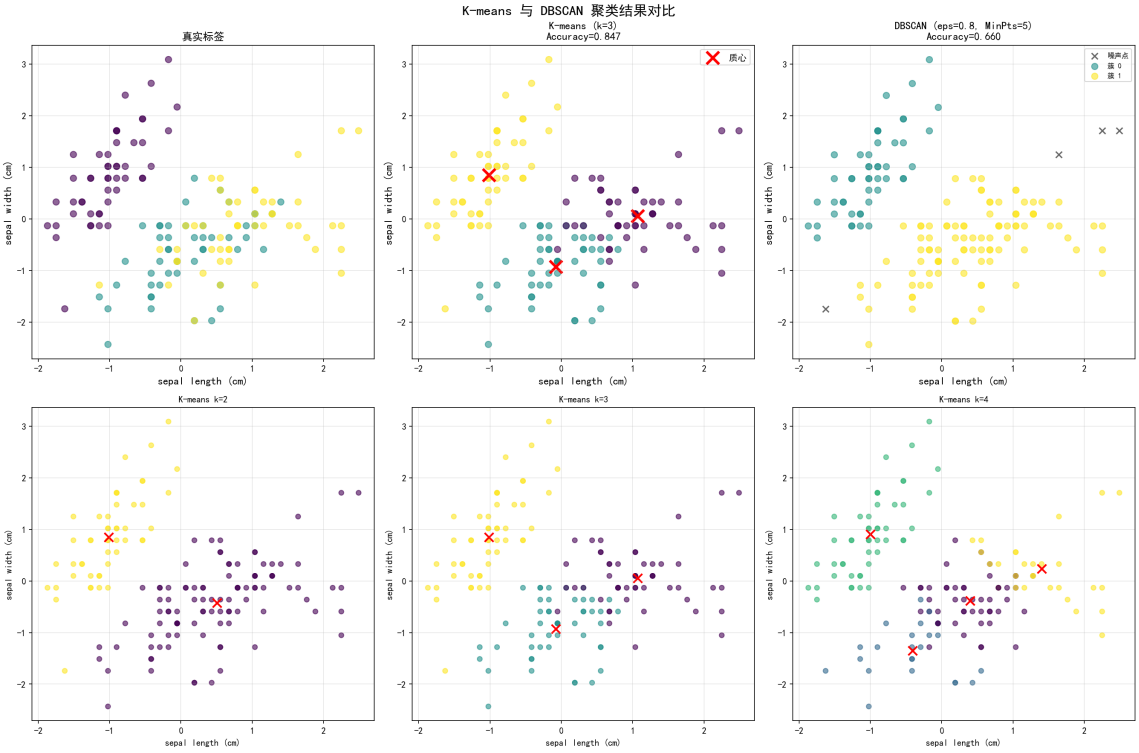


图 3: K-means 与 DBSCAN 聚类结果综合对比

4.4 定量评估结果

表 1: K-means 不同 k 值的评估指标

k 值	准确率	轮廓系数	Calinski-Harabasz 指数
2	0.6667	0.5818	251.3493
3	**0.8467**	0.4566	241.3156
4	0.6933	0.4151	206.2286
5	0.6133	0.3576	167.8065

表 2: DBSCAN 不同 ϵ 值的评估指标（MinPts=5）

ϵ	簇数	噪声点数	准确率	轮廓系数	Calinski-Harabasz 指数
0.2	0	150	0.0000	0.0000	0.0000
0.5	2	34	0.5600	**0.6559**	**343.9681**
0.8	2	4	**0.6600**	0.5979	277.6510
1.2	2	1	0.6600	0.5868	256.8506

4.5 结果分析

1. **K-means 特点**：算法简单高效，适合球形簇；当 $k = 3$ 时准确率最高（84.67%）。缺点是假设簇为凸形，需要预先指定 k 值。

2. **DBSCAN 特点**：能识别任意形状的簇和噪声点，不需要预先指定簇数。缺点是对参数敏感，在 Iris 数据集上准确率较低（最高 66.00%）。

3. **对比结论**：K-means 在 Iris 数据集上表现更好，因为 Iris 数据的三个类别大致呈球形分布。DBSCAN 更适合处理具有密度差异和任意形状簇的数据集。

5 实验总结

本实验通过手动实现 K-means 和 DBSCAN 两种聚类算法，在 Iris 数据集上进行了系统的对比分析。实验结果表明：K-means 在 $k = 3$ 时达到最佳准确率 84.67%，验证了其对球形簇的有效性；DBSCAN 能够有效识别噪声点，但在 Iris 数据集上准确率较低。两种算法各有优劣，实际应用中应根据数据特点选择合适的算法——若数据簇呈球形分布且已知簇数，优先选择 K-means；若数据包含噪声或簇形状不规则，优先选择 DBSCAN。